Software Testing and Quality Assurance
Module 4: Test Automation and Testing for Specialized Environment

Need of automation

Reduction of testing effort. In verification and validation strategies, numerous test case design methods have been studied. Test cases for a complete software may be hundreds of thousands or more in number. Executing all of them manually takes a lot of testing effort and time. Thus, execution of test suits through software tools greatly reduces the amount of time required.

Reduces the testers' involvement in executing tests. Sometimes executing the test cases takes a long time. Automating this process of executing the test suit will relieve the testers to do some other work, thereby increasing the parallelism in testing efforts.

Facilitates regression testing. As we know, regression testing is the most time-consuming process. If we automate the process of regression testing, then testing effort as well as the time taken will reduce as compared to manual testing. Avoids human mistakes Manually executing the test cases may incorporate errors in the process or sometimes, we may be biased towards limited test cases while checking the software. Testing tools will not cause these problems which are introduced due to manual testing.

Reduces overall cost of the software. As we have seen, if testing time increases, the cost of the software also increases. But due to testing tools, time and therefore cost can be reduced to a greater level as testing tools ease the burden of the test case production and execution.

Simulated testing. Load performance testing is an example of testing where the real-life situation needs to be simulated in the test environment. Sometimes, it may not be possible to create the load of a number of concurrent users or large amounts of data in a project. Automated tools, on the other hand, can create millions of concurrent virtual users/data and effectively test the project in the test environment before releasing the product.

Internal testing. Testing may require testing for memory leakage or checking the coverage of testing. Automation tools can help in these tasks quickly and accurately, whereas doing this manually would be cumbersome, inaccurate, and time-consuming.

Test enablers. While development is not complete, some modules for testing are not ready. At that time, stubs or drivers are needed to prepare data, simulate the environment, make calls, and then verify results. Automation reduces the effort required in this case and becomes essential.

Test case design. Automated tools can be used to design test cases also. Through automation, better coverage can be guaranteed, than if done manually.

Guidelines for automated testing

Consider building a tool instead of buying one, if possible. It may not be possible every time. But if the requirement is small and sufficient resources allow, then go for building the tool instead of buying, after weighing the pros and cons. Whether to buy or build a tool requires management commitment, including budget and resource approvals.

Test the tool on an application prototype While purchasing the tool, it is important to verify that it works properly with the system being developed. However, it is not possible as the system being developed is often not available. Therefore, it is suggested that if possible, the development team can build a system prototype for evaluating the testing tool.

Not all the tests should be automated. Automated testing is an enhancement of manual testing, but it cannot be expected that all tests on a project can be automated. It is important to decide which parts need automation before going for tools.

Some tests are impossible to automate, such as verifying a printout. It has to be done manually.

Select the tools according to organizational needs. Do not buy the tools just for their popularity or to compete with other organizations. Focus on the needs of the organization and know the resources (budget, schedule) before choosing the automation tool.

Use proven test-script development techniques. Automation can be effective if proven techniques are used to produce efficient, maintainable, and reusable test scripts. The following are some hints:

1. Read the data values from either spreadsheets or tool-provided data pools, rather than being hard-coded into the test-case script because this prevents test cases from being reused. Hard-coded values should be replaced with variables and whenever possible read data from external sources.

2. Use modular script development. It increases maintainability and readability of the source code.

3. Build a library of reusable functions by separating the common actions into a shared script library usable by all test engineers.

4. All test scripts should be stored in a version control tool. Automate the regression tests whenever feasible Regression testing consumes a lot of time. If tools are used for this testing, the testing time can be reduced to a greater extent. Therefore, whenever possible, automate the regression test cases.

Categorization of testing tools

A single tool may not cover the whole testing process, therefore, a variety of testing tools are available according to different needs and users.

Static and dynamic testing tools
These tools are based on the type of execution of test cases, namely static and dynamic, as discussed in software testing techniques:

Static testing tools. For static testing, there are static program analysers which scan the source program and detect possible faults and anomalies. These static tools parse the program text, recognize the various sentences, and detect the following:

- Statements are well-formed.
- Inferences about the control flow of the program.
- Compute the set of all possible values for program data.

Static tools perform the following types of static analysis:

Control flow analysis. This analysis detects loops with multiple exits and entry points and unreachable code.

Data use analysis. It detects all types of data faults.

Interface analysis. It detects all interface faults. It also detects functions which are never declared and never called or function results that are never used.

Path analysis. It identifies all possible paths through the program and unravels the program's control.

Dynamic testing tools. These tools support the following:

Dynamic testing activities.

- Many times, systems are difficult to test because several operations are being performed concurrently. In such

cases, it is difficult to anticipate conditions and generate representative test cases. Automated test tools enable the test team to capture the state of events during the execution of a program by preserving a snapshot of the conditions. These tools are sometimes called program monitors. The monitors perform the following functions:

- List the number of times a component is called or line of code is executed. This information about the statement or path coverage of their test cases is used by testers.
- Report on whether a decision point has branched in all directions, thereby providing information about coverage.
- Report summary statistics providing a high-level view of the percentage of statements, paths, and branches that have been covered by the collective set of test cases run. This information is important when test objectives are stated in terms of coverage.

Testing activity tools

These tools are based on the testing activities or tasks in a particular phase of the SDLC. Testing activities can be categorized as:

- Reviews and inspections
- Test planning
- Test design and development
- Test execution and evaluation

The testing tools based on these testing tasks are:

Tools for review and inspections. Since these tools are for static analysis on many items, some tools are designed to work with specifications but there are far too many tools available that work exclusively with code. In this category, the following types of tools are required:

- **Complexity analysis tools**. It is important for testers that complexity is analysed so that testing time and resources can be estimated. The complexity analysis tools analyse the areas of complexity and provide indication to testers.
- **Code comprehension**. These tools help in understanding dependencies, tracing program logic, viewing graphical representations of the program, and identifying the dead code. All these tasks enable the inspection team to analyse the code extensively.

Tools for test planning. The types of tools required for test planning are:

- Templates for test plan documentation
- Test schedule and staffing estimates
- Complexity analyser

Tools for test design and development. Below are the types of tools required.

Test data generator. It automates the generation of test data based on a user defined format. These tools can populate a database quickly based on a set of rules, whether data is needed for functional testing, data-driven load testing, or performance testing.

Test case generator. It automates the procedure of generating the test cases. But it works with a requirement management tool which is meant to capture requirements information. The test case generator uses the information provided by the requirement management tool and creates the test cases. The test cases can also be generated with the information provided by the test engineer regarding the previous failures that have been discovered by him. This information is entered into this tool and it becomes the knowledge-based tool that uses the knowledge of historical figures to generate test cases.

Test execution and evaluation tools. The types of tools required for test execution and evaluation are:

Capture/playback tools: These tools record events (including keystrokes, mouse activity, and display output) at the time of running the system and place the information into a script. The tool can then replay the script to test the system.

Coverage analysis tools. These tools automate the process of thoroughly testing the software and provide a quantitative measure of the coverage of the system being tested. These tools are helpful in the following:

- Measuring structural coverage which enables the development and test teams to gain insight into the effectiveness of tests and test suites
- Quantifying the complexity of the design
- Help in specifying parts of the software which are not being covered
- Measure the number of integration tests required to qualify the design
- Help in producing integration tests
- Measuring the number of integration tests that have not been executed
- Measuring multiple levels of test coverage, including branch, condition, decision/condition, multiple conditions, and path coverage.

Memory testing tools. These tools verify that an application is properly using its memory resources. They check whether an application is:

- Not releasing memory allocated to it
- Overwriting/overreading array bounds
- Reading and using uninitialized memory

Test management tools Test management tools try to cover most of the activities in the testing life cycle. These tools may cover planning, analysis, and design. Some test management tools such as Rational's TestStudio are integrated with requirement and configuration management and defect tracking tools, in order to simplify the entire testing life cycle.

Network-testing tools. There are various applications running in the client-server environments. However, these applications pose new complexity to the testing effort and increases potential for errors due to inter-platform connectivity. Therefore, these tools monitor, measure, test, and diagnose performance across an entire network including the following:

- Cover the performance of the server and the network
- Overall system performance
- Functionality across server, client, and the network

Performance testing tools. There are various systems for which performance testing is a must but this becomes a tedious job in real-time systems. Performance testing tools help in measuring the response time and load capabilities of a system.

Selection of testing tools

The selection may depend on several factors. What are the needs of the organization; what is the project environment; what is the current testing methodology; all these factors should be considered when choosing testing tools. Some guidelines to be followed by selecting a testing tool are given below.

Match the tool to its appropriate use. Before selecting the tool, it is necessary to know its use. A tool may not be a general one or may not cover many features. Rather, most of the tools are meant for specific tasks. Therefore, the tester needs to be familiar with both the tool and its uses in order to make a proper selection.

Select the tool to its appropriate SDLC phase. Since the methods of testing changes according to the SDLC phase, the testing tools also change. Therefore, it is necessary to choose the tool according to the SDLC phase, in which testing is to be done.

Select the tool to the skill of the tester. The individual performing the test must select a tool that conforms to his skill level. For example, it would be inappropriate for a user to select a tool that requires programming skills when the user does not possess those skills.

Select a tool which is affordable. Tools are always costly and increase the cost of the project. Therefore, choose the tool which is within the budget of the project. Increasing the budget of the project for a costlier tool is not desired. If the tool is

under utilization, then added cost will have no benefits to the project. Thus, once you are sure that a particular tool will really help the project, then only go for it otherwise it can be managed without a tool also.

Determine how many tools are required for testing the system. A single tool generally cannot satisfy all test requirements. It may be possible that many test tools are required for the entire project. Therefore, assess the tool as per the test requirements and determine the number and type of tools required.

Select the tool after examining the schedule of testing. First, get an idea of the entire schedule of testing activities and then decide whether there is enough time for learning the testing tool and then performing automation with that tool. If there is not enough time to provide training on the tool, then there is no use of automation.

Data driven testing

Data-driven testing (DDT) is a software testing methodology where test data is stored in external data sources like spreadsheets, databases, or CSV files, rather than hard coded into the test case, allowing the same test logic to be executed multiple times with different sets of data.

When using a data-driven testing methodology, we create test scripts or test flows to be executed together with their related data sets in a test automation framework or tool.

The goal of this is to separate test logic from test data, creating reusable tests where testers can run the same test with multiple sets of data, and thereby improving test coverage and reducing maintenance.

Data-driven testing is used in conjunction with automated testing, which can read data from external sources and execute test cases automatically.

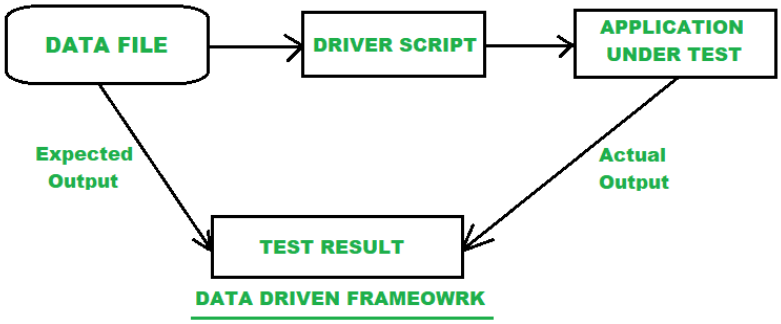
Types of data driven testing

- **Keyword-Driven Testing:** A data-driven method called "keyword-driven testing" keeps the logic of test scripts apart from the data. Keywords are used in test scripts; a data table lists the inputs and anticipated outcomes for each keyword. As a result, test cases can be changed freely without affecting the scripts.
- **Excel-Driven Testing:** Excel spreadsheets are used in Excel-driven testing to specify and arrange test data. Test scripts may run tests with various sets of data and are easy to maintain because they read data from Excel files.
- **Negative Testing:** In a data-driven approach, negative testing consists of evaluating the application's capacity to manage unexpected or erroneous data. Test cases are made to verify how the system reacts to values entered incorrectly.
- **XML Testing Driven:** XML files are used to describe test data in XML-driven testing. To run tests with various data sets, test scripts read data from XML files. This method works especially well with complex data structures.

Working of data driven testing

Suppose taking an application for login credentials both username and password are required.

- If both the username and password are correct then login successfully, and it will enter into the home page.
- Another case is that when the username is correct, but the password is wrong then the login action will fail i.e. showing an Invalid credential and will not allow entering into the home page.
- Next when the username is incorrect and the password is correct then the login fails.
- The next thing the logic says is that it is a control flow process for checking the conditions and functions.
- The test scripts are taken in the final position and an automation code is implemented for the conditions. The test scripts are working like functions and actions.



Advantages

- **Reusability of Code:** Data-driven testing allows the reusability of code. It improves test coverage.
- **Data Values:** In the case of regression testing, the test application allows the set of data values.
- **Test and Verification of data:** In data-driven testing mainly both the test and verification data are ordered in a single file and separated from some test logic. This type of testing generates a clear-cut idea and logic of the test cases from the test data for the developers and testers.
- Changes in the test script do not affect the test data as well as the test process.
- **Time-Saving:** Different tools generate the test data automatically and a large volume of test data is taken to save time when necessary.
- **Flexibility:** Less maintenance is required as well as it provides flexibility in application maintenance.
- **Re-using functions:** Several functions and actions can be reused in many test cases present in data-driven testing.
- **Reduce Redundancy:** It helps in reducing redundancy and unnecessary duplication of test scripts.

Disadvantages

- **Dependency on Automation Team Skills:** One of the biggest drawbacks is that the quality of the test depended upon the automation team skills i.e. being implemented.
- **Scripting Language Expertise:** Data-driven testing requires great knowledge and expertise in the scripting language.
- **Execution Time:** When the amount of data is more for validation it takes so much time to execute.
- **Maintenance:** In this type, maintenance plays a big issue in the code complexity and difficulty of understanding the logic.
- **High-Level Technical Skills Requirement:** More documentation and high-level technical skills are required. Another thing is that the tester should learn the entire new scripting language.

Keyword driven testing

The keyword-driven testing is based upon a keyword-driven framework that defines functional automation testing and is categorized into four different parts: test steps for test cases, objects, actions, and data sets. It is a software engineering technique or approach that is used in functional automation testing that's why it's called a type of functional automation testing. A table format is used for defining keywords or action words in this technique which is why called table-driven testing and the keywords or action words are defined for each function/method in this technique which is why called Action word-based testing. In this type of testing a table or spreadsheet format is used to define keywords or action words for every instruction that is under the execution stage. Different types of actions and test data are combinable and by providing these inputs the driver script plays a vital role in producing the output of the test result accordingly.

E.g.: Llogin to “geeksforgeeks” website – Keyword “Sign in” will be used for automation purposes for login to the Website, to test the login function or function related to the same.

Examples of keywords

- **Excel Sheet:** Storing the keywords for test cases.
- **Function Library:** It contains functions for business false(e.g., login button).
- **Data Sheets:** Storing the test data which is used in the application.
- **Object Repository:** using the object repository based on the framework which you want.
- **Test Scripts:** Developing the test scripts for manual test cases or a single driver script.

Phases of keyword driven testing

1. Design and Development Phase: In the design and development phase, the set of actions briefly explains the mentioned keywords that are designed. The number of actions performed in this type is assigned with a single keyword and accordingly, it is working sequentially.

2. Implementation Phase: In the implementation stage, the final stage of execution may be performed in manual or automatic ways and sometimes it may be performed in both of the ways based upon the situation. The whole commands are executed in a set very precisely that can be determined in the primary stage of the function execution.

Advantages

- One of the major advantages of this testing is that functional testers can plan test automation even before the application is ready.
- For considering with no programming knowledge the test cases could be developed.
- Another key advantage is that it is independent of any specific programming language or any other tool.
- Most automation tools available are compatible with this testing technique.

Disadvantages

- The bigger disadvantage is that it is a time-consuming process to develop the keywords and the functionalities of the testing.
- The obstacles are driven by the technical testers. The keywords may prevent the testers from preventing their technological idea and years of experience during the test.

Testing web based systems

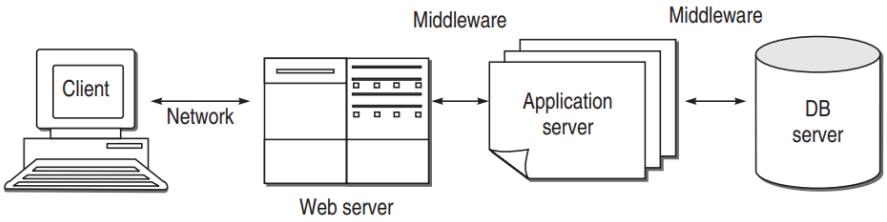
The web-based software system consists of a set of web pages and components that interact to form a system which executes using web server(s), network, HTTP, and a browser, and in which user input affects the state of the system. Thus, web-based systems are typical software programs that operate on the Internet, interacting with the user through an Internet browser. Some other terms regarding these systems are given below:

Web page is the information that can be viewed in a single browser window.

A **website** is a collection of web pages and associated software components that are related semantically by content and syntactically through links and other control mechanisms. Websites can be dynamic and interactive.

A **web application** is a program that runs as a whole or in part on one or more web servers and that can be run by users through a website. Web applications require the presence of a web server in simple configurations and multiple servers in more complex settings. Such applications are called **web-based applications**. Similar applications, which may operate independent of any servers and rely on operating system services to perform their functions, are termed **web-enabled applications**. These days, with the integration of technologies used for the development of such applications, there is a thin line separating web-based and web-enabled applications, so we collectively refer to both of them as web applications.

Web technology evolution



First generation/2-tier web system: The web systems initially were based on a typical client-server model. The client is a web browser that people use to visit websites. The websites are on different computers, the servers and the HTML files are sent to the client by a software package called a web server. HTML files contain JavaScripts, which are small pieces of code that are interpreted on the client.

Modern 3-tier and n-tier architecture: To get quality attributes, such as security, reliability, availability, scalability, and functionality, an application server has been introduced wherein most of the software has been moved to a separate computer. On large websites, the application server is actually a collection of application servers that operate in parallel. The application servers typically interact with one or more database servers, often running a commercial database. Web and application servers often are connected by middleware.

Web-based systems have a different nature as compared to traditional systems. Due to the environment difference and challenges of dynamic behaviour, complexity and diversity also makes the testing of these systems a challenge. These systems need to be tested not only to check whether it does what it is designed to do but also to evaluate how well it appears on the (different) web browsers. Moreover, they need to be tested for various quality parameters which are a must for these systems like security, usability, etc. Hence, a lot of effort is required for test planning and test designing.

Test cases should be written covering the different scenarios not only of the functional usage but also the technical implementation environment conditions such as network speeds, screen resolution, etc. For example, an application may work fine on Broadband Internet users but may perform miserably for users with dial-up connections. Web applications are known to give errors on slow networks, whereas they perform well on high-speed connections. Web pages don't render correctly for certain situations but work fine with others. Images may take longer to download for slower networks and the end user perception of the application may not be good.

There may be a number of navigation paths possible through a web application. Therefore, all these paths must be tested. Along with multiple navigation paths, users may have varying backgrounds and skills. Testing should be performed keeping in view all the possible categories of users in view. This becomes an issue in usability testing.

Another issue of great concern is the security testing of web applications. There are two cases. For Intranet-based applications, there are no such threats on the application. However, in case of Internet-based applications, the users may need to be authenticated and security measures may have to be much more stringent. Test cases need to be designed to test the various scenarios and risks involved.

Traditional software must be tested on different platforms, or it may fail on some platforms. Similarly, web-based software must be tested with all the dimensions which are making it diverse. For example, users may have different browsers while accessing the applications. This aspect also needs to be tested under compatibility testing. If we test the application only on Internet Explorer, then we cannot ensure that it works well on Netscape or other browsers. Because these browsers may not only render pages differently but also have varying levels of support for client side scripting languages such as Java Script.

The strategy for testing web-based systems is the same as for other systems, i.e. verification and validation. Verification

largely considers the checking of analysis and design models which have been described above. Various types of testing derive from the design models only. However, the quality parameters are also important factors which form the other types of testing. We discuss these various types of web testing in the following sections.

Quality aspects

Reliability. It is an expected feature that the system and servers are available at any time with the correct results. It means that the web systems must have high availability to be a reliable system. If one web software fails, the customer should be able to switch to another one. Thus, if web software is unreliable, websites that depend on the software will lose customers and the businesses may lose large amounts of money.

Performance. Performance parameters for web systems must also be tested. If they are not tested, they may even lead to the total failure of the system. Performance testing is largely on the basis of load testing, response time, download time of a web page, or transactions performed per unit time.

Security. Since web systems are based on the Internet, there are possibilities of any type of intrusion in these systems. Thus, security of web applications is another challenge for the web applications developers as well as for the testing team. Security is critical for e-commerce websites.

Tests for security are often broken down into two categories: testing the security of the infrastructure hosting the web application and testing for vulnerabilities of the web application. Some of the points that should be considered for infrastructure are firewalls and port scans. For vulnerabilities, there is user authentication, cookies, etc. Data collected must be secured internally. Users should not be able to browse through the directories in the server. A cookie is a text file on a user's system that identifies the user. Cookies must always be encrypted and must not be available to other users.

Usability. This is another major concern while testing a web system. The users are real testers of these systems. If they feel comfortable in using them; then the systems are considered of high quality. Thus, usability also becomes an issue for the testers. The outlook of the system, navigation steps, all must be tested keeping the large categories of users in mind. Moreover, the users expect to be able to use the websites without any training. Thus, the software must flow according to the users' expectations, offer only needed information, and when needed, should provide navigation controls that are clear and obvious.

Scalability. With the evolution of fast technology changes, the industry has developed new software languages, new design strategies, and new communication and data transfer protocols, to allow websites to grow as needed. Therefore, designing and building web software applications that can be easily scaled is currently one of the most interesting and important quality issues in software design. Scalability is defined as the web application's capacity to sustain the number of concurrent users and/or transactions, while sustaining sufficient response times to its users. To test scalability, web traffic loads must be determined in order to obtain the threshold requirement for scalability. Sometimes, existing traffic levels are used to simulate the load.

Challenges

Web-based software is different as compared to traditional systems. Therefore, we must understand the environment of their creation and then test them. Keeping in view the environment and behaviour of web-based systems, we face many challenges while testing them. These challenges become issues and guidelines when we perform testing of these systems. Some of the challenges and quality issues for web-based system are:

Diversity and complexity. Web applications interact with many components that run on diverse hardware and software platforms. They are written in diverse languages and they are based on different programming approaches such as procedural, OO, and hybrid languages such as Java Server Pages (JSPs). The client side includes browsers, HTML, embedded scripting languages, and applets. The server side includes CGI, JSPs, Java Servlets, and .NET technologies.

They all interact with diverse back-end engines and other components that are found on the web server or other servers.

Dynamic environment. The key aspect of web applications is its dynamic nature. The dynamic aspects are caused by uncertainty in the program behaviour, changes in application requirements, rapidly evolving web technology itself, and other factors. The dynamic nature of web software creates challenges for the analysis, testing, and maintenance for these systems. For example, it is difficult to determine statically the application's control flow because the control flow is highly dependent on user input and sometimes in terms of trends in user behaviour over time or user location. Not knowing which page an application is likely to display hinders statically modeling the control flow with accuracy and efficiency.

Very short development time. Clients of web-based systems impose very short development time, compared to other software or information systems projects. For e.g. an e-business system, sports website, etc.

Continuous evolution. Demand for more functionality and capacity after the system has been designed and deployed to meet the extended scope and demands, i.e. scalability issues.

Compatibility and interoperability. As discussed above, there may also be compatibility issues that make web testing a difficult task. Web applications often are affected by factors that may cause incompatibility and interoperability issues. The problem of incompatibility may exist on both the client as well as the server side. The server components can be distributed to different operating systems. Various versions of browsers running under a variety of operating systems can be there on the client side. Graphics and other objects on a website have to be tested on multiple browsers. If more than one browser will be supported, then the graphics have to be visually checked for differences in the physical appearance. The code that executes from the browser also has to be tested. There are different versions of HTML. They are similar in some ways but they have different tags which may produce different features.

Security testing

Today, the web applications store more vital data and the number of transactions on the web has increased tremendously with the increasing number of users. Therefore, in the Internet environment, the most challenging issue is to protect the web applications from hackers, crackers, spoofers, virus launchers, etc. Through security testing, we try to ensure that data on the web applications remain confidential, i.e. there is no unauthorized access. Security testing also ensures that users can perform only those tasks that they are authorized to perform.

In a web application, the risk of attack is multifold, i.e. it can be on the web software, client-side environment, network communications, and server-side environments. Therefore, web application security is particularly important because these are generally accessible to more users than desktop applications. Often, they are accessed from different locations, using different systems and browsers, exposing them to different security issues, especially external attacks. It means that web applications must be designed and developed such that they are able to nullify any attack from outside. Therefore, this issue is also related to testing of web applications in terms of security. We need to design the test cases such that the application passes the security test.

Security test plan

Security testing can be divided into two categories: testing the security of the infrastructure hosting the web application and testing for vulnerabilities of the web application. Firewalls and port scans can be the solution for security of infrastructure. For vulnerabilities, user authentication, restricted and encrypted use of cookies, data communicated must be planned. Moreover, users should not be able to browse through the directories in the server.

Planning for security testing can be done with the help of some threat models. These models may be prepared at the time of requirement gathering and test plan can also be prepared correspondingly. These threat models will help in incorporating security issues in designing and later can also help in security testing.

Find all the component interfaces for performing security testing on a component. This is because most of the security bugs can be found on the interfaces only. The interfaces are then prioritized according to their level of vulnerability. The high-priority interfaces are tested thoroughly by injecting mutated data to be accessed by that interface in order to check the security. While performing security testing, the testers should take care that they do not modify the configuration of the application or the server, services running on the server, and existing user or customer data hosted by the application.

Various threat types and their corresponding test cases

Unauthorized user/Fake identity/Password cracking. When an unauthorized user tries to access the software by using fake identity, then security testing should be done such that any unauthorized user is not able to see the contents/data in the software.

Cross-site scripting (XSS). When a user inserts HTML/client-side script in the user interface of a web application and this insertion is visible to other users, it is called cross-site scripting (XSS). Attackers can use this method to execute malicious scripts or URLs on the victim's browser. Using cross-site scripting, attackers can use scripts like JavaScript to steal user cookies and information stored in the cookies. To avoid this, the tester should additionally check the web application for XSS.

Buffer overflows. Buffer overflow is another big problem when handling memory allocation if there is no overflow check on the client software. Due to this problem, malicious code can be executed by the hackers. In the application, check the buffer overflow module and the different ways of submitting a range of lengths to the application.

URL manipulation. There may be chances that communication through HTTP is also not safe. The web application uses the HTTP GET method to pass information between the client and the server. The information is passed in parameters in the query string. Again, the attacker may change some information in query string passed from GET request so that he may get some information or corrupt the data. When one attempts to modify the data, it is known as fi ddling of data. The tester should check if the application passes important information in the query string and design the test cases correspondingly. Write the test cases such that a general user tries to modify the private information.

SQL injection. Hackers can also put some SQL statements through the web application user interface into some queries meant for querying the database. In this way, he can get vital information from the server database. Even if the attacker is successful in crashing the application, from the SQL query error shown on the browser, the attacker can get the information they are looking for. Design the test cases such that special characters from user inputs should be handled/escaped properly in such cases.

Denial of service. When a service does not respond, it is denial of service. There are several ways that can make an application fail. For example, heavy load put on the application, distorted data that may crash the application, overloading of memory, etc. Design the test cases considering all these factors

Navigation testing

Navigation testing involves verifying that all links, menus, and buttons within an application work as intended. It ensures that users are guided efficiently through their journey and are not confused or stuck while interacting with the site or app.

Typical use cases include e-commerce platforms, blogs, social media websites, and apps where users must navigate through various pages to complete tasks such as purchasing, finding content, or interacting with features. This type of testing is relevant for QA engineers, UI/UX designers, product managers, and web developers, as they aim to improve user experiences and minimize errors caused by broken or confusing navigation.

Navigation testing is essential for the following reasons:

1. **Improving User Experience**: Seamless navigation helps users find what they need without frustration, leading to higher satisfaction rates and fewer drop-offs. A positive user experience encourages users to return and engage

further.

2. **Reducing Errors**: Faulty or broken navigation links can lead to dead ends, preventing users from completing desired actions. This can cause frustration, leading to a loss of trust and potential business. Navigation testing ensures all navigation paths are functional.
3. **Improving SEO**: Good navigation positively affects search engine optimization (SEO). Search engines rank websites based on how easily they can crawl and index pages. Proper navigation helps search engine bots navigate and index content better, improving a site's visibility in search engine results.

Below are the different types of navigation testing:

1. **Categorization Testing**: Categorization testing focuses on how well content is organized within the website or application. It assesses whether users can easily find information based on how categories and subcategories are structured. Effective categorization ensures users can quickly navigate to their desired sections, enhancing the overall experience.
2. **Information Architecture Testing**: Information architecture (IA) testing evaluates the organization and labeling of information on a site. It examines whether the structure makes sense to users and whether they can intuitively navigate the content they seek. A well-designed IA reduces cognitive load and helps users find relevant information without confusion.
3. **Layout and Functionality Testing**: This type of testing assesses the visual arrangement and functionality of navigation elements, such as menus, buttons, and links. It ensures that users can interact effectively with navigation items and respond appropriately to user actions. Proper layout and functionality contribute to a seamless user experience, making it easier for users to explore the site.

Performance testing

Web applications' performance is also a big issue in today's busy Internet environment. The user wants to retrieve the information as soon as possible without any delay. This assumes the high importance of performance testing of web applications. Is the application able to respond in a timely manner or is it ready to take the maximum load or beyond that? These questions imply that web applications must also be tested for performance. Performance testing helps the developer to identify the bottlenecks in the system and can be rectified.

In performance testing, we evaluate metrics like response time, throughput, and resource utilization against desired values. Using the results of this evaluation, we are able to predict whether the software is in a condition to be released or requires improvement before it is released. Moreover, we can also find the bottlenecks in the web application. Bottlenecks for web applications can be code, database, network, peripheral devices, etc.

Performance parameters: Performance parameters, against which the testing can be performed, are:

- **Resource utilization.** The percentage of time a resource (CPU, Memory, I/O, Peripheral, Network) is busy.
- **Throughput.** The number of event responses that have been completed over a given interval of time.
- **Response time.** The time lapsed between a request and its reply.
- **Database load.** The number of times a database is accessed by a web application over a given interval of time.
- **Scalability.** The ability of an application to handle additional workload, without adversely affecting performance, by adding resources such as processor, memory, and storage capacity.
- **Round-trip time.** How long does the entire user-requested transaction take, including connection and processing time?

Types of Performance Testing: Performance tests are broadly divided into following categories:

Load testing. Can the system sustain at times of peak load? The site should handle many simultaneous user requests, large input data from users, simultaneous connection to DB, heavy load on specific pages, etc. When we need to test the

application with these types of loads, then load testing is performed on the system. It focuses on determining or validating performance characteristics of the system when subjected to workloads and load volumes anticipated during production operations. It refers to how much load (the best example of load in a web application is how many concurrent users) you can put on the web application and it will still serve flawlessly.There are a few types of load testing which can be performed. We can perform capacity testing to determine the maximum load the web service can handle before failing. Capacity testing reveals the web services' ultimate limit. We may also perform scalability testing to determine how effectively the web service will expand to accommodate an increasing load.

Stress testing. Generally, stress refers to stretching the system beyond its specification limits. Web stress testing is performed to break the site by giving stress and check how the system reacts to stress and how the system recovers from crashes. It focuses on determining or validating performance characteristics of the system when subjected to conditions beyond those anticipated during production operations. It also tests the performance of the system under stressful conditions, such as limited memory, insufficient disk space, or server failure. These tests are designed to determine under what conditions an application will fail, how it will fail, and how gracefully it may recover from the failure. Some examples of graceful failure are: the system saves state at the time of failure and does not crash suddenly; On restarting it, the system recovers from the last good state; the system shows meaningful error messages to the user.

Testing agile based software

Agile Testing is a Type of Software Testing that follows the principles of agile software development to test the software application. All members of the project team, along with the special experts and testers, are involved in agile testing. Agile testing is not a separate phase, and it is carried out with all the development phases, which are requirements, design, coding, and test case generation.
Agile testing is a flexible and dynamic process that runs continuously throughout each iteration of the Software Development Life Cycle (SDLC). The main focus for Agile testers is customer satisfaction, verifying that the product meets the needs and expectations of the users.

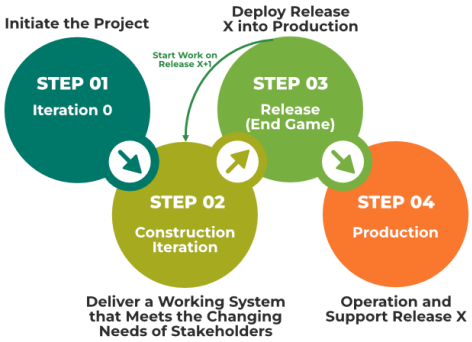
Agile testing principles

1. **Shortening feedback iteration:** In Agile Testing, the testing team gets to know the product development and its quality for each and every iteration. Thus continuous feedback minimizes the feedback response time, and the fixing cost is also reduced.
2. **Involvement of all members:** Agile testing involves each and every member of the development team and the testing team. It includes various developers and experts.
3. **Clean code:** The defects that are detected are fixed within the same iteration. This ensures clean code at any stage of development.
4. **Constant response:** Agile testing helps to deliver responses or feedback on an ongoing basis. Thus, the product can meet the business needs.
5. **Customer satisfaction:** In agile testing, customers are exposed to the product throughout the development process. Throughout the development process, the customer can modify the requirements, and update the requirements and the tests can also be changed as per the changed requirements.

Agile testing methodologies

1. **Test-Driven Development (TDD)**: In TDD, tests are written before writing the actual code. This approach involves three steps: writing a unit test, coding to pass the test, and then refactoring the code. It verifies that the code is always tested and improved in small, manageable steps.
2. **Exploratory Testing**: Here, testers are free to explore the software as they see fit, without following predefined test scripts. This method helps uncover unknown risks and bugs by allowing testers to test the software in creative and flexible ways.

3. **Acceptance Test-Driven Development (ATDD)**: ATDD involves the customer, developers, and testers working together to define the requirements and potential challenges before coding begins. This collaborative effort reduces the chance of errors and helps build software that meets customer needs from the start.
4. **Extreme Programming (XP)**: XP is focused on delivering high-quality software that meets customer needs. It involves practices that emphasize customer involvement, simplicity, and frequent releases to ensure the final product aligns with customer expectations.
5. **Dynamic Software Development Method (DSDM)**: DSDM is an Agile framework for delivering projects. It provides a set of principles for developers, users, and testers to collaborate and build systems that meet the needs of the business.



Mobile application testing

Mobile application testing is a process by which application software developed for handheld mobile devices is tested for its functionality, usability and consistency. Mobile application testing can be an automated or manual type of testing. Mobile applications either come pre-installed or can be installed from mobile software distribution platforms.

There are broadly 2 kinds of testing that take place on mobile devices:

1. **Hardware testing:** The device including the internal processors, internal hardware, screen sizes, resolution, space or memory, camera, radio, Bluetooth, WIFI etc. This is sometimes referred to as simple "Mobile Testing".
2. **Software or application testing:** The applications that work on mobile devices and their functionality are tested. It is called the "Mobile Application Testing" to differentiate it from the earlier method. Even in the mobile applications, there are few basic differences that are important to understand:
    a) **Native apps**: A native application is created for use on a platform like mobile and tablets.
    b) **Mobile web apps** are server-side apps to access website/s on mobile using different browsers like chrome, Firefox by connecting to a mobile network or wireless network like WIFI.
    c) **Hybrid apps** are combinations of native app and web app. They run on devices or offline and are written using web technologies like HTML5 and CSS.

Challenges

1. Wide varieties of mobile devices like Samsung, Apple and Nokia.
2. Different range of mobile devices with different screen sizes and hardware configurations like hard keypad, virtual keypad (touch screen) and trackball etc.
3. Different mobile operating systems like Android, Symbian, Windows, Blackberry and IOS.
4. Different versions of the operating system like iOS 5.x, iOS 6.x, BB5.x, BB6.x etc.
5. Different mobile network operators like GSM(Global System for Mobile Communication) and CDMA(Code Division Multiple Access).
6. Frequent updates (like android- 4.2, 4.3, 4.4, iOS-5.x, 6.x) – with each update a new testing cycle is recommended to make sure no application functionality is impacted.

Types of mobile app testing

1. **Compatibility testing–** Testing of the application in different mobiles devices, browsers, screen sizes and OS versions according to the requirements.
2. **Interface testing–** Testing of menu options, buttons, bookmarks, history, settings, and navigation flow of the application.

3. **Services testing–** Testing the services of the application online and offline.
4. **Low level resource testing**: Testing of memory usage, auto deletion of temporary files, local database growing issues known as low level resource testing.
5. **Performance testing–** Testing the performance of the application by changing the connection from 2G, 3G to WIFI, sharing the documents, battery consumption, etc.
6. Testing the performance and behaviour of the application under certain conditions such as low battery, bad network coverage, low available memory, simultaneous access to application's server by several users and other conditions. Performance of an application can be affected from two sides: application's server side and client's side. Performance testing is carried out to check both.
7. **Operational testing**– Testing of backups and recovery plan if battery goes down, or data loss while upgrading the application from store.
8. **Security Testing–** Testing an application to validate if the information system protects data or not.

Mobile application testing strategy

1) Selection of the devices – Analyze the market and choose the devices that are widely used. (This decision mostly relies on the clients. The client or the app builders consider the popularity factor of a certain devices as well as the marketing needs for the application to decide what handsets to use for testing.)

2) **Emulators** – The use of these is extremely useful in the initial stages of development, as they allow quick and efficient checking of the app. Emulator is a system that runs software from one environment to another environment without changing the software itself. It duplicates the features and work on real system. Types of Mobile Emulators:

1. Device Emulator- provided by device manufacturers
2. Browser Emulator- simulates mobile browser environments.
3. Operating systems Emulator- Apple provides emulators for iPhones, Microsoft for Windows phones and Google Android phones

3) After a satisfactory level of development is complete for the mobile app, you could move to test on the physical devices for a more real life scenario based testing.

4) Consider cloud computing based testing: Cloud computing is basically running devices on multiple systems or networks via the Internet where applications can be tested, updated and managed. For testing purposes, it creates the web based mobile environment on a simulator to access the mobile app.

5)Automation vs. Manual testing

- If the application contains new functionality, test it manually.
- If the application requires testing once or twice, do it manually.
- Automate the scripts for regression test cases. If regression tests are repeated, automated testing is perfect for that.
- Automate the scripts for complex scenarios which are time consuming if executed manually.

Two kinds of automation tools are available to test mobile apps:

1. Object based mobile testing tools– automation by mapping elements on the device screen into objects. This approach is independent of screen size and mainly used for Android devices.
2. Eg:- ranorex, jamo solution
3. Image based mobile testing tools– create automation scripts based on screen coordinates of elements.
4. Eg:- Sikuli, Egg Plant, RoutineBot

6) Network configuration is also a necessary part of mobile testing. It's important to validate the application on different networks like 2G, 3G, 4G or WIFI.

Sample test cases for testing a mobile app

In addition to functionality based test cases, Mobile application testing requires special test cases which should cover following scenarios.

1. Battery usage– It's important to keep a track of battery consumption while running application on the mobile devices.
2. Speed of the application- the response time on different devices, with different memory parameters, with different network types etc.
3. Data requirements – For installation as well as to verify if the user with limited data plan will able to download it.
4. Memory requirement– again, to download, install and run
5. Functionality of the application– make sure the application is not crashing due to network failure or anything else.